

BÁO CÁO TỔNG HỢP TIÊU CHUẨN IEEE VÀ SO SÁNH TOÀN DIỆN VỚI NESTJS (v01)

Dự án: Cowdi English Platform (cowdi.net) Ngày lập: Tháng 09/2026 Phiên bản: v01 Đơn vị thực hiện: Đội ngũ Kỹ thuật & Kiến trúc Hệ thống Cowdi

I. TỔNG QUAN

Báo cáo này phân tích hệ thống **Cowdi English** dưới góc nhìn của **Viện Kỹ sư Điện và Điện tử Quốc tế (IEEE - Institute of Electrical and Electronics Engineers)** và tổ chức chuẩn hóa liên hợp **ISO/IEC/IEEE**.

Đồng thời, báo cáo cung cấp một **ma trận so sánh chuyên sâu, đa chiều giữa cấu trúc hiện tại (React SPA + Express.js + MySQL) và nền tảng kiến trúc doanh nghiệp NestJS** — bao gồm cả khía cạnh tuân thủ tiêu chuẩn kỹ thuật lẫn các phương diện thực tế: hiệu năng, tài nguyên, chi phí phát triển, độ phức tạp và khả năng mở rộng.

II. DANH MỤC TIÊU CHUẨN IEEE ĐANG ÁP DỤNG TRÊN WEBSITE COWDI

Dù không phải là một thiết bị phần cứng viễn thông, nền tảng phần mềm của Cowdi tuân thủ và kế thừa nhiều tiêu chuẩn cốt lõi thuộc **Hội đồng Tiêu chuẩn Kỹ nghệ Phần mềm của IEEE (IEEE Computer Society - SESC)**:

CÁC TIÊU CHUẨN IEEE ÁP DỤNG TRÊN HỆ THỐNG COWDI

2. MÔ TẢ KIẾN TRÚC | ISO/IEC/IEEE 42010:2011 (Decoupled Viewpoints: Client / API / DB)

4. MÔI TRƯỜNG THỰC THI | IEEE 1003.1 (POSIX Interface: Node.js Runtime, Process, Signals)

1. IEEE 754-2019 — Chuẩn Số học Dấu phẩy động (Floating-Point Arithmetic)

Đặc tả: Chuẩn quốc tế quy định định dạng biểu diễn số học nhị phân và các phép toán dấu phẩy động 64-bit (Double Precision - `binary64`). **Ứng dụng thực tế tại Cowdi:**

- Thuật toán tính toán điểm kinh nghiệm **XP**, tỷ lệ hoàn thành bài học (%), tỷ lệ ghi nhớ từ vựng theo đường cong Spaced Repetition.
- Xử lý tỷ lệ thắng/thua trong các trận đấu PvP **Duel 1v1** và công thức tính điểm leo Rank trên Bảng xếp hạng.
- Toàn bộ các phép toán trên cả trình duyệt (V8 Engine của Chrome/Edge) và Node.js server đều tuân thủ nhất quán theo IEEE 754, đảm bảo không có sai lệch dữ liệu giữa Client và Server.

2. ISO/IEC/IEEE 42010:2011 — Chuẩn Mô tả Kiến trúc Phần mềm (Architecture Description)

Đặc tả: Tiêu chuẩn quốc tế quy định nguyên tắc phân tách rõ ràng giữa các góc nhìn kiến trúc (**Architectural Viewpoints**) và mối quan tâm độc lập (**Separation of Concerns**). **Ứng dụng thực tế tại Cowdi:**

- Cowdi áp dụng triệt để kiến trúc phân tách **Decoupled Architecture**:
- Client Presentation Viewpoint*: Giao diện người dùng độc lập trong `src/` (React SPA + PWA).
- Offline & Service Viewpoint*: Bộ nhớ đệm và xử lý ngầm tại Service Worker (`src/sw.js`).
- Business & API Controller Viewpoint*: Tầng định tuyến và kiểm soát truy cập tại `server/routes/` và `server/middleware/`.

- *Persistence Data Viewpoint*: Tầng lưu trữ bền vững tại `server/config/database.js` (MySQL Pool).

3. ISO/IEC/IEEE 12207:2017 — Chuẩn Quy trình Vòng đời Phần mềm (Software Life Cycle Processes)

Đặc tả: Quy định các quy trình kỹ thuật từ phát triển, đóng gói đến phát hành và bảo trì hệ thống phần mềm. **Ứng dụng thực tế tại Cowdi:**

- Quy trình tự động hóa chu kỳ đóng gói (Build Pipeline): Sử dụng `Vite` để phân tách mã nguồn (Code-splitting), nén dữ liệu (Tree-shaking, Minification), sinh PWA icons tự động (`scripts/generate-pwa-icons.mjs`) và phân định rõ ràng giữa môi trường Development và Production.

4. IEEE 1003.1 (POSIX) — Chuẩn Giao diện Hệ điều hành Di động

Đặc tả: Chuẩn hóa giao diện lập trình ứng dụng (API) tương tác giữa phần mềm và hệ điều hành nhân Unix/Linux. **Ứng dụng thực tế tại Cowdi:**

- Backend Node.js của Cowdi chạy trên hệ điều hành Linux (aaPanel/Nginx) tuân thủ hoàn toàn chuẩn POSIX: cơ chế biến môi trường (`process.env`), quản lý tiến trình không đồng bộ (Asynchronous Event Loop), tín hiệu kết thúc an toàn (Graceful Shutdown) và luồng đọc ghi file (Streams/Buffer).

5. IEEE 802.3 (Ethernet) & IEEE 802.11 (Wi-Fi)

Toàn bộ các kết nối truyền dữ liệu API và tải tài nguyên web giữa thiết bị người học và máy chủ Cowdi đều chạy trên hạ tầng mạng tuân thủ chuẩn IEEE 802.11 (khi kết nối Wi-Fi trên điện thoại/laptop) và IEEE 802.3 (mạng cáp quang tốc độ cao tại máy chủ).

III. ĐỐI CHIẾU MỨC ĐỘ TUÂN THỦ CHUẨN IEEE: COWDI HIỆN TẠI VS. NESTJS

Mã hiệu Tiêu chuẩn IEEE	Tên chuẩn hóa	Cấu trúc Cowdi Hiện tại (Express)	Nền tảng NestJS
IEEE 754-2019	Số học thực dấu phẩy động (XP, Rank, %)	Tuân thủ 100%	Tuân thủ 100%
ISO/IEC/IEEE 42010	Tách rời góc nhìn kiến trúc phần mềm	Tuân thủ (Tách rời thư mục thủ công)	Tuân thủ chặt chẽ (Ép theo Module/Controller/Service)
ISO/IEC/IEEE 12207	Quy trình vòng đời đóng gói sản phẩm	Tuân thủ (Build pipeline Vite)	Tuân thủ (CLI build Webpack/SWC)
ISO/IEC/IEEE 1016:2009	Mô tả thiết kế phần mềm dựa trên Component & Interface	⚠ Chưa hoàn chỉnh (Dùng JS động, chưa ép Interface/DTO)	✅ Tuân thủ tuyệt đối (100% TypeScript DTO & Entities)
ISO/IEC/IEEE 730:2014	Quy chuẩn Kiểm thử & Đảm bảo Chất lượng (SQA)	⚠ Chưa có (Chưa viết bộ Unit Test tự động)	✅ Có sẵn bộ khung (Tích hợp sẵn Jest Unit Test & E2E Test)

IV. SO SÁNH CHI TIẾT CÁC PHƯƠNG DIỆN NGOÀI TIÊU CHUẨN KỸ THUẬT

Để có cái nhìn toàn diện phục vụ quyết định đầu tư công nghệ, dưới đây là so sánh chi tiết trên **6 phương diện vận hành và kinh doanh thực tế**:

SO SÁNH CÁC PHƯƠNG DIỆN THỰC TẾ: COWDI (EXPRESS) VS. NESTJS

2. TỐC ĐỘ PHÁT TRIỂN (TTM) | Express: Rất nhanh, ít boilerplate VS. NestJS: Chậm hơn ở giai đoạn đầu

4. KHẢ NĂNG MỞ RỘNG (SCALE) | Express: Dễ rớt khi >100 API VS. NestJS: Chuẩn hóa, code đồng bộ

6. TYPE-SAFETY (AN TOÀN) | Express: Rủi ro runtime error VS. NestJS: Bắt lỗi ngay lúc code

1. Hiệu năng & Tiêu thụ Tài nguyên Phần cứng (Performance & Footprint)

Cowdi Hiện tại (Express.js):

- **Tối ưu RAM:** Khởi động chỉ tiêu tốn khoảng **30–45MB RAM**. Đây là lợi thế cực lớn giúp máy chủ giá rẻ (VPS 1GB-2GB RAM) vẫn có thể chạy mượt mà cả Nginx, MySQL và Node.js.
- **Khởi động siêu tốc:** Khởi động lại dịch vụ chỉ mất **dưới 1 giây**.

NestJS:

- **Tiêu thụ RAM lớn hơn:** Do phải nạp toàn bộ bộ khung Dependency Injection (IoC Container), Metadata Reflection và các module TypeScript, NestJS tốn từ **120–180MB RAM** khi vừa khởi động.
- **Khởi động chậm hơn:** Mất từ **3–5 giây** để biên dịch và nạp container.

2. Tốc độ Phát triển Ban đầu (Time-to-Market)

Cowdi Hiện tại (Express.js):

- Cực kỳ linh hoạt. Khi cần thêm một tính năng mới (ví dụ: Tạo mã giới thiệu bạn bè **Invite**), lập trình viên chỉ cần tạo 1 file route tầm 40 dòng là xong.
- Không mất thời gian cấu hình khai báo kiểu dữ liệu, phù hợp tối đa cho giai đoạn **khởi nghiệp (Startup / MVP)** cần thử nghiệm nhanh thị trường.

NestJS:

- Tốn nhiều thời gian tạo "Boilerplate code". Một tính năng đơn giản cũng bắt buộc phải tạo ít nhất 4 file: `invite.module.ts`, `invite.controller.ts`, `invite.service.ts`, `create-invite.dto.ts`.
 - Tốc độ ra mắt tính năng ban đầu sẽ chậm hơn khoảng 30–40%.
-

3. Đường cong Học tập & Nguồn lực Nhân sự (Learning Curve & Hiring)

Cowdi Hiện tại (Express.js):

- Express là thư viện phổ biến số 1 thế giới Node.js. Bất kỳ lập trình viên JavaScript nào từ thực tập sinh đến chuyên nghiệp đều có thể đọc hiểu và sửa code ngay trong ngày đầu tiên tiếp cận.

NestJS:

- Đường cong học tập rất dốc. Đòi hỏi lập trình viên phải thành thạo:
 - Ngôn ngữ **TypeScript nâng cao** (Generics, Decorators, Utility Types).
 - Tư duy lập trình Hướng đối tượng (**OOP**) và nguyên lý **SOLID**.
 - Khái niệm **Inversion of Control (IoC) & Dependency Injection**.
 - Thư viện lập trình bất đồng bộ **RxJS** (trong trường hợp xử lý Interceptor).
 - Chi phí tuyển dụng hoặc đào tạo đội ngũ cho NestJS tốn kém hơn đáng kể.
-

4. Khả năng Bảo trì & Mở rộng Quy mô (Maintainability & Enterprise Scalability)

Cowdi Hiện tại (Express.js):

- Khi hệ thống còn nhỏ (dưới 50 API), cấu trúc rất thanh thoát. Tuy nhiên, khi hệ thống phình to lên 200–300 API và có 5–10 lập trình viên cùng can thiệp, nếu không có sự kỷ luật thép, code rất dễ biến thành "**Spaghetti code**", các hàm phụ thuộc lẫn nhau gây khó khăn cho việc bảo trì.

NestJS:

- Đây chính là "sân nhà" của NestJS. Do có cấu trúc khuôn mẫu bắt buộc, 10 lập trình viên viết code sẽ cho ra sản phẩm đồng nhất 100%. Rất dễ dàng tái cấu trúc (Refactoring), chuyển đổi sang kiến trúc **Microservices** hoặc tách module độc lập khi lượng người dùng tăng đột biến.
-

5. Khả năng Xử lý Thời gian thực (Real-time Multiplayer cho Tính năng Duel)

Cowdi Hiện tại (Express.js):

- Tính năng thách đấu Duel hiện tại đang xử lý theo mô hình REST Polling hoặc cần cấu hình thủ công thư viện ngoài nếu muốn dùng Socket.io.

NestJS:

- Tích hợp sẵn kiến trúc **WebSockets Gateways** (`@WebSocketGateway()`). Cung cấp sẵn mô hình Room, Broadcast, và Adapter chuẩn giúp việc phát triển tính năng đấu trí PvP trực tiếp giữa 2 người học (nhìn thấy đối thủ trả lời theo từng giây) trở nên cực kỳ dễ dàng, chuyên nghiệp và chịu tải cao.
-

6. Độ an toàn Dữ liệu & Bắt lỗi Thời gian thực (Type-Safety & Defensiveness)

Cowdi Hiện tại (Express.js):

- Dùng JavaScript thuần. Nếu lập trình viên vô tình gọi một thuộc tính không tồn tại (`user.profile.age`), server có thể bị crash đột ngột với lỗi `TypeError: Cannot read property of undefined` ngay trên Production.
- NestJS:**
- Sử dụng 100% TypeScript và cơ chế **Class-Validator DTO**. Mọi lỗi sai kiểu dữ liệu đều bị phát hiện ngay khi gõ phím (Compile-time). Dữ liệu rác người dùng gửi lên API sẽ bị chặn tự động ở cổng vào mà không thể lọt vào tầng xử lý logic.

V. MA TRẬN TỔNG HỢP SO SÁNH ĐA PHƯƠNG DIỆN

Phương diện đánh giá	Cowdi Hiện tại (Express.js)	Chuyển đổi sang NestJS	Nhận định chiến lược
Tuân thủ chuẩn IEEE (Kiến trúc)	Khá (Kế thừa IEEE 42010, 12207)	Xuất sắc (Chuẩn hóa IEEE 42010, 1016, 730)	NestJS chuẩn hóa bài bản hơn
Tiết kiệm RAM & Chi phí Server	🟢 Rất tốt (< 50MB RAM)	🟡 Trung bình (120 - 180MB RAM)	Cowdi hiện tại tối ưu chi phí hơn
Tốc độ đưa sản phẩm ra thị trường	🟢 Nhanh (Phát triển thần tốc)	🟡 Chậm hơn (Nhiều file mẫu bắt buộc)	Cowdi hiện tại linh hoạt hơn
Dễ tuyển dụng & tiếp cận code	🟢 Rất dễ (JS phổ thông)	🔴 Khó hơn (Đòi hỏi chuyên sâu TS/OOP)	Cowdi hiện tại dễ tìm nhân sự hơn
Độ ổn định cho dự án siêu lớn	🟡 Trung bình (Cần kỷ luật cao)	🟢 Xuất sắc (Ép khuôn doanh nghiệp)	NestJS vượt trội cho team lớn
Hỗ trợ Duel Realtime (WebSockets)	🟡 Thủ công	🟢 Mạnh mẽ có sẵn	NestJS tối ưu cho game thi đấu
Phát hiện lỗi sớm (Type-Safety)	🔴 Runtime check	🟢 Compile-time check	NestJS an toàn hơn về dữ liệu

VI. KẾT LUẬN & LỘ TRÌNH KHUYẾN NGHỊ CHO COWDI ENGLISH

1. Giai đoạn Hiện tại (Nên giữ nguyên cấu trúc Express hiện tại):

- Cấu trúc Backend hiện tại của Cowdi đang vận hành **rất ổn định, nhẹ nhàng, chi phí máy chủ thấp và hoàn toàn tuân thủ các chuẩn quốc tế quan trọng** (IEEE 754, ISO/IEC/IEEE 42010, RFC OAuth2/JWT).

- Chưa cần thiết phải đập đi xây lại bằng NestJS vào thời điểm này để tránh lãng phí nguồn lực và làm chậm tiến độ phát triển tính năng phục vụ người học.

2. Lộ trình Nâng cấp Khuyến nghị (Khi đạt quy mô > 50.000 người dùng):

- **Bước 1:** Bổ sung bộ kiểm thử tự động (Unit Test với Jest) vào backend hiện tại để hoàn thiện tiêu chuẩn **ISO/IEC/IEEE 730**.
- **Bước 2 (Khi nâng cấp tính năng Duel):** Bóc tách riêng phân hệ **Duel PvP Realtime** thành một Microservice độc lập viết bằng **NestJS WebSockets Gateway**. Khi đó Cowdi sẽ tận dụng được sức mạnh lớn nhất của NestJS mà không làm xáo trộn toàn bộ hệ thống hiện có.